Testing Guide: React Hydration Error Fix

Overview

This document provides instructions for testing the React Hydration Error fix applied to the signup page.

What Was Fixed

Problem

The signup page (`/signup`) was experiencing a React Hydration Error:
- **Error Message**: “Hydration failed because the initial UI does not match what was rendered on the server”
- **Affected Components**: Clerk’s `<SignUp>` and `<Portal>` components
- **Root Cause**: Server/client rendering mismatch due to:
1. `useSearchParams()` hook without Suspense boundary
2. `useTheme()` hook causing theme mismatch between server and client

Solution Applied

1. **Wrapped component in Suspense boundary** - Required for `useSearchParams()` in Next.js 13+
2. **Added mounted state check** - Prevents theme-related hydration mismatches
3. **Proper theme resolution** - Handles `system` theme correctly
4. **Loading states** - Shows spinner during initial mount and Suspense fallback

Test Checklist

✅ Prerequisites

- [x] Next.js development server is available
- [x] Clerk authentication is configured (optional for testing page load)
- [x] Node.js and npm installed

🧪 Test Cases

Test 1: Basic Page Load

```
# Start the dev server
cd /home/ubuntu/mindous
npm run dev

# Navigate to: http://localhost:3000/signup
```

Expected Result:
- ✅ Page loads without errors
- ✅ No hydration warnings in browser console
- ✅ Brief loading spinner may appear
- ✅ Clerk SignUp component renders

Check Browser Console:
- ❌ Should NOT see: “Hydration failed”
- ❌ Should NOT see: “Text content did not match”
- ❌ Should NOT see: “Warning: Expected server HTML to contain”

Test 2: With URL Parameters

```
Test URLs:
1. http://localhost:3000/signup?email=test@example.com
2. http://localhost:3000/signup?email=test@example.com&token=abc123
3. http://localhost:3000/signup?payment=success&email=test@example.com
```

Expected Result:
- ✅ Email field is pre-filled (if Clerk is configured)
- ✅ Payment success alert shows (when `payment=success`)
- ✅ No hydration errors in console

Test 3: Theme Switching

```
1. Open page in light theme
2. Switch to dark theme (if theme toggle available)
3. Refresh page
4. Switch to system theme
```

Expected Result:
- ✅ Theme changes apply correctly
- ✅ No hydration errors after theme switches
- ✅ Clerk component theme matches app theme

Test 4: Production Build

```
npm run build
npm start
# Navigate to: http://localhost:3000/signup
```

Expected Result:
- ✅ Build completes without errors
- ✅ Page works in production mode
- ✅ No hydration errors

Verification Steps

Step 1: Check Browser Console

1. Open browser DevTools (F12)
2. Go to Console tab
3. Navigate to `/signup`
4. Look for errors

Pass Criteria:
- No “Hydration” errors

- No “Text content did not match” warnings
- May see Clerk API errors (expected if not configured)

Step 2: Check Network Tab

1. Open Network tab in DevTools
2. Reload the page
3. Check for status codes

Pass Criteria:
- Initial page load returns 200
- Component loads successfully
- Clerk resources load (or fail gracefully if not configured)

Step 3: Test User Flow

1. Navigate to signup page
2. (If Clerk configured) Try to sign up
3. Check redirect behavior

Pass Criteria:
- Signup form appears
- Form is interactive
- No console errors during interaction

Common Issues & Solutions

Issue: Still seeing hydration errors

Solution:
- Clear browser cache and `.next` folder
- Restart dev server
- Check if other components are causing issues

```
rm -rf .next
npm run dev
```

Issue: Page shows infinite loading

Solution:
- Check browser console for errors
- Verify Clerk configuration
- Check if `useSearchParams()` is working

Issue: Theme not applying correctly

Solution:
- Verify ThemeProvider is in root layout
- Check localStorage for theme value
- Clear browser storage and refresh

Developer Notes

Key Changes Made

```
// Before:
export default function SignUpPage() {
  const { theme } = useTheme();
  const searchParams = useSearchParams();
  // ... component code
}

// After:
function SignUpContent() {
  const { theme, systemTheme } = useTheme();
  const searchParams = useSearchParams();
  const [isMounted, setIsMounted] = useState(false);

  useEffect(() => {
    setIsMounted(true);
  }, []);

  if (!isMounted) {
    return <LoadingSpinner />;
  }
  // ... component code
}

export default function SignUpPage() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <SignUpContent />
    </Suspense>
  );
}
```

Why This Fixes Hydration Errors

1. **Suspense Boundary**: Next.js docs require Suspense for `useSearchParams()`
   - Prevents SSR/CSR mismatch for dynamic route params
   - Provides loading state during param resolution

2. **Mounted State**: Prevents theme mismatch
   - Server doesn't have access to localStorage
   - Client waits for mount before rendering theme-dependent UI
   - Ensures identical initial render

3. **Theme Resolution**: Handles system theme properly
   - Resolves `system` to actual theme value
   - Prevents undefined theme causing mismatch

Success Criteria

✅ **Fix is successful if:**
- No hydration errors in browser console
- Page loads and displays correctly
- URL parameters work as expected

- Theme switching works without errors
- Production build works correctly

❌ **Fix needs revision if:**
- Hydration errors still appear
- Page doesn’t load properly
- Components don’t render
- Build fails

Additional Resources

- Next.js useSearchParams Documentation (https://nextjs.org/docs/app/api-reference/functions/use-search-params)
- React Hydration Errors (https://react.dev/reference/react-dom/client/hydrateRoot#hydrating-server-rendered-html)
- Clerk Next.js Integration (https://clerk.com/docs/quickstarts/nextjs)

Contact & Support

If issues persist:
1. Check `HYDRATION_FIX_SUMMARY.md` for detailed explanation
2. Review browser console errors
3. Verify all dependencies are installed
4. Check Next.js and React versions compatibility